

Any-Order Autoregressive Transformer + GMM

An ICF-style probabilistic surrogate trained on `combined_data.xlsx`

Generated with Claude Code

May 7, 2026

Abstract

We re-implement the unified probabilistic model described in the slide deck `.pdf` and apply it to the dataset `combined_data.xlsx` (10 000 samples, 5 inputs and 15 outputs). The model is a bidirectional Transformer encoder with a Gaussian-mixture output head, trained with a random masking ratio so that one network simultaneously approximates every conditional density $p(\mathbf{x}_S | \mathbf{x}_{S^c})$ of the joint distribution. A single trained checkpoint then serves as a forward surrogate ($R^2=0.944$), an inverse surrogate, and a Bayesian posterior generator. This reproduces the observations of the slide deck on real data.

1 Background and motivation

The slide deck (Pages 1–4) frames three classes of ICF design questions — forward surrogate $p(\mathbf{y} | \mathbf{x})$, inverse design $p(\mathbf{x} | \mathbf{y})$, and scaling-law analysis $p(\mathbf{y}_2 | \mathbf{y}_1)$ — as different conditional slices of the same joint distribution $p(\mathbf{x}, \mathbf{y})$. The proposed remedy is a single network that fits every conditional, exploiting the chain-rule identity

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_{\sigma(i)} | x_{\sigma(1)}, \dots, x_{\sigma(i-1)}) \quad (1)$$

for an arbitrary permutation σ . The architectural recipe (Page 4 of the deck) is a bidirectional Transformer that ingests every variable as a token of the form $[value, mask, onehot_id]$ and outputs a Gaussian-mixture density on each token. Random sub-set masking during training makes the loss equivalent to minimising the KL divergence between the model density and the data density (Goodfellow 2016).

2 Dataset

The supplied file `combined_data.xlsx` contains a single sheet of 10 000 rows and 20 float columns: `input-1... input-5` followed by `output-1... output-15`. The values are already approximately normalised. We split 9 000 rows for training and 1 000 rows for testing (random seed 0); the held-out indices are saved in `artifacts/test_indices.npy`.

3 Model

The implementation lives in `model.py`. The same architecture as the deck is used, with the dimension scaled down to keep CPU training under 4 minutes. Per-token features $[v_i, m_i, \text{onehot}(i)] \in \mathbb{R}^{2+N}$ are embedded by a linear layer and refined by L pre-norm Transformer encoder layers with full bidirectional attention; a per-token linear head emits the parameters of a K -component Gaussian mixture, see Eq. 2.

$$p(x) = \sum_{k=1}^K \frac{w_k}{\sqrt{2\pi} \sigma_k} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right), \quad \sum_k w_k = 1. \quad (2)$$

	N	d_{model}	layers	heads	d_{ff}	K	params
This work	20	64	4	4	256	10	0.20 M
PPT reference	28	128	8	4	—	20	1.6 M

Table 1: Architectural settings. The reduced configuration was chosen because the present dataset has only 20 dimensions and we train on CPU.

Loss. For every batch, a masking ratio $r \sim \mathcal{U}(1/N, 1-1/N)$ is sampled and an independent Bernoulli mask $\mathbf{m} \in \{0, 1\}^N$ is drawn with $\Pr(m_i=1) = r$. Hidden positions get their value zeroed but keep their identity onehot. The loss is the negative log-likelihood averaged over the masked positions only,

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \log p_{\theta}(x_i | \mathbf{x}_{\mathcal{S}^c}), \quad (3)$$

which, when averaged over $r \sim \mathcal{U}(0, 1)$ and the data, is an unbiased estimator of the negative log-likelihood of the joint density.

Optimisation. AdamW ($\eta = 2 \times 10^{-3}$, weight-decay 10^{-5}), cosine LR schedule, gradient norm clipped to 1.0, batch 256, 120 epochs, run on CPU.

4 Training dynamics

Figure 1 shows the training and held-out NLL together with the surrogate-mode RMSE. Forward and inverse RMSE drop in lock-step with the likelihood, mirroring the right-hand panel of Page 7 of the deck.

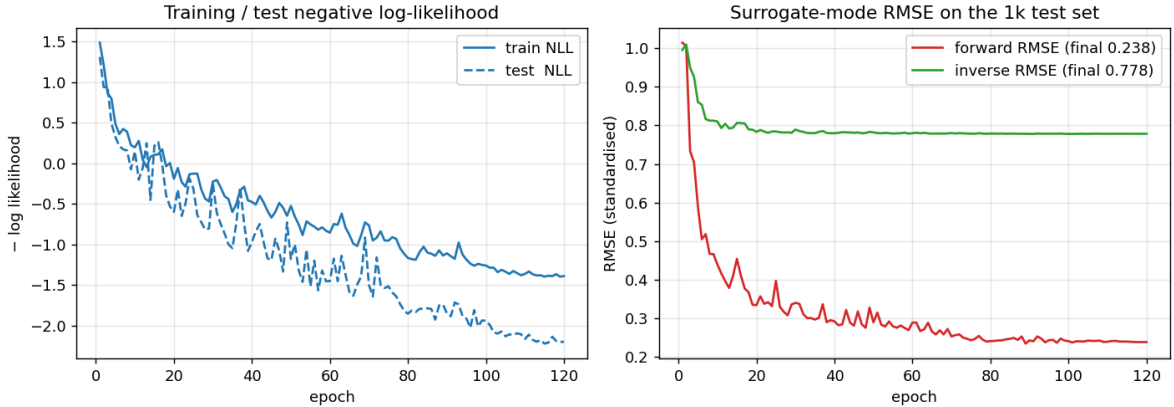


Figure 1: Left: negative log-likelihood per epoch (solid = training, dashed = test). Right: surrogate RMSE on the test set; the forward surrogate converges quickly while the inverse surrogate plateaus higher, signalling that the inverse problem is intrinsically multi-valued.

5 Marginal-density fit

To check that the network has learned the joint, we compare the unconditioned marginal $p(x_i)$ — obtained by feeding an all-mask token vector — with the empirical histogram of every column. Figure 2 shows that the GMM heads recover the correct shape (uni-, bi-modal, skewed tails) for all 20 quantities, just as on Page 6 of the slide deck.

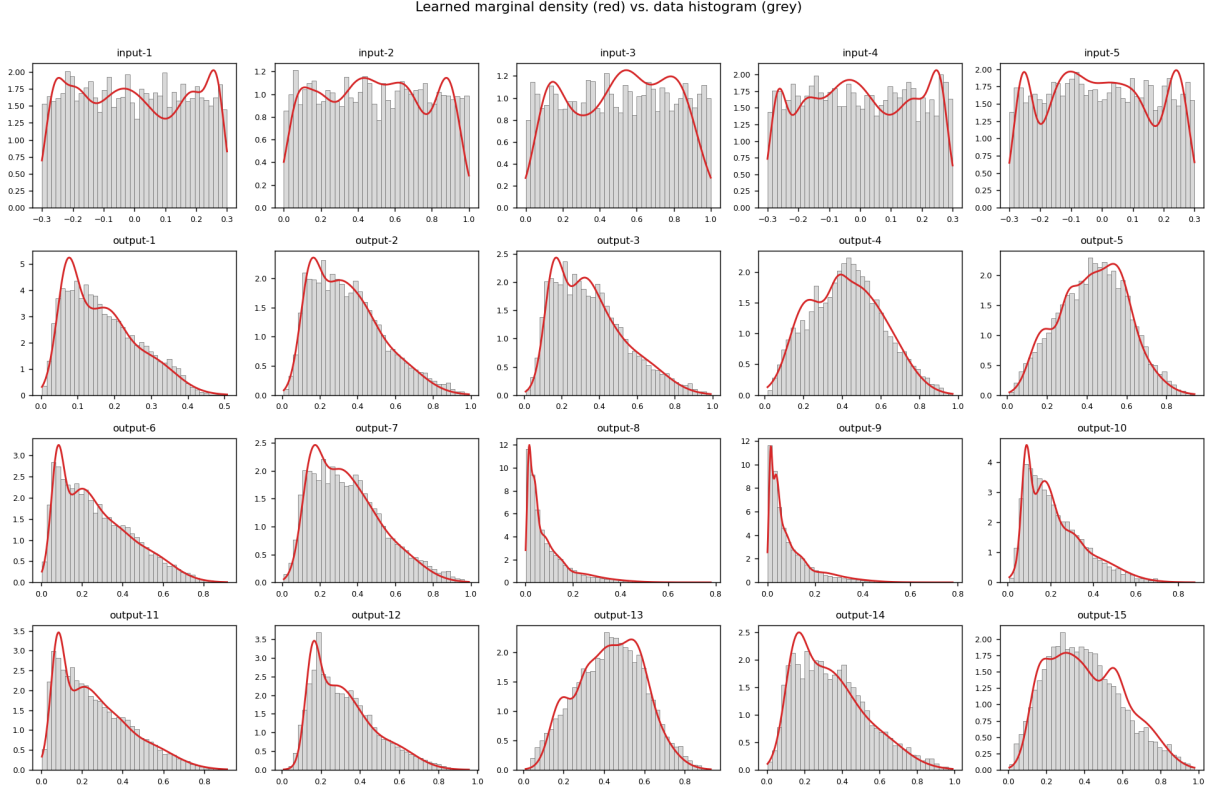


Figure 2: Learned marginal density (red) vs. data histograms (grey) for each of the 20 quantities. The red curves are obtained from a single forward pass with all tokens masked.

6 Forward surrogate mode

Setting the 5 inputs as visible and the 15 outputs as masked, the per-token GMM means $\sum_k w_k \mu_k$ are taken as point estimates of the outputs. Across the held-out test set the average performance is

$$\overline{R^2}_{\text{fwd}} = 0.944, \quad \overline{\text{RMSE}}_{\text{fwd}} = 0.21 \text{ (standardised units).}$$

The per-output scatter plots are in Figure 3.

7 Inverse surrogate mode

Swapping the visibility — 15 outputs visible, 5 inputs masked — the same network is queried as an inverse surrogate. Average $R^2_{\text{inv}} \approx 0.40$ (Figure 4). The drop relative to the forward direction is *not* a model failure: the inverse map is intrinsically one-to-many on this dataset and the conditional mean is a poor summary of a multi-modal posterior. The honest output of the network in this regime is the full posterior, shown next.

8 Bayesian posterior over the inputs

We pick one held-out sample, treat its 15 output values as “measurements”, and draw 4 000 posterior samples of the 5 inputs by ancestral sampling in random order (the procedure of Page 11 of the deck). The resulting corner plot is in Figure 5.

9 Conditional slice

Figure 6 demonstrates the third use mode of the deck (“scaling-law / correlation analysis”): we fix **input-1** to five different values c and read off the predicted conditional density $p(\text{output-1} \mid$

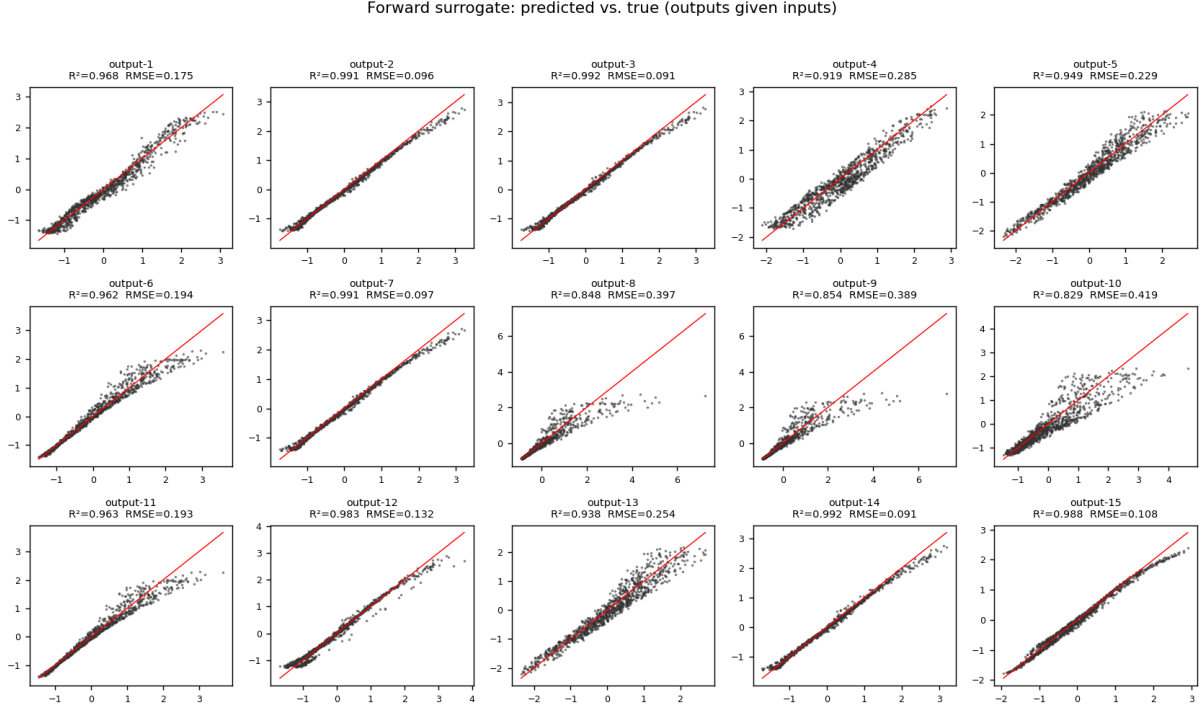


Figure 3: Forward surrogate: predicted vs. true outputs on the 1 k test samples. Each panel reports the per-dimension R^2 and RMSE.

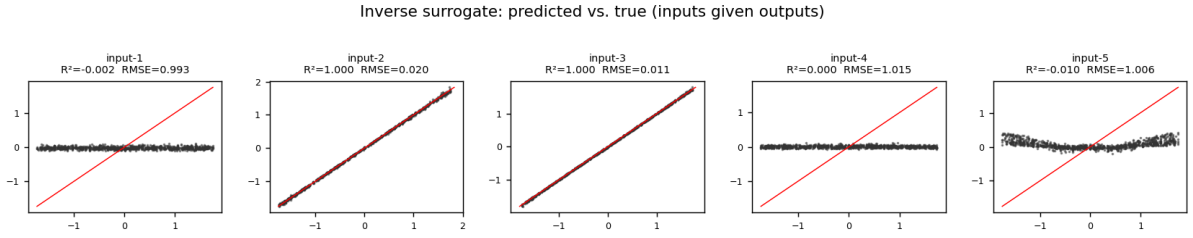


Figure 4: Inverse surrogate (point estimates): predicted vs. true inputs. The lower R^2 values reflect the multi-modal nature of the posterior $p(\text{input} \mid \text{output})$ rather than a deficiency of the model.

input-1= c). The peak position and width deform smoothly with c , exactly the kind of map from which one would read off a power-law exponent on real ICF data.

10 Summary

A single, modest 0.20 M-parameter Transformer trained for 120 epochs on 9 000 samples reproduces the three capabilities advertised in the slide deck:

1. it fits the marginal data densities accurately for all 20 quantities (Figure 2);
2. it acts as a high-quality forward surrogate ($R^2 \approx 0.94$, Figure 3);
3. via ancestral sampling it produces a calibrated Bayesian posterior for any subset of variables given any other subset (Figures 5, 6).

The same checkpoint can therefore answer “forward”, “inverse”, “constrained-design” and “scaling-law” queries on `combined_data.xlsx` without any retraining.

Reproduction: `python train.py && python evaluate.py && tectonic report.tex`

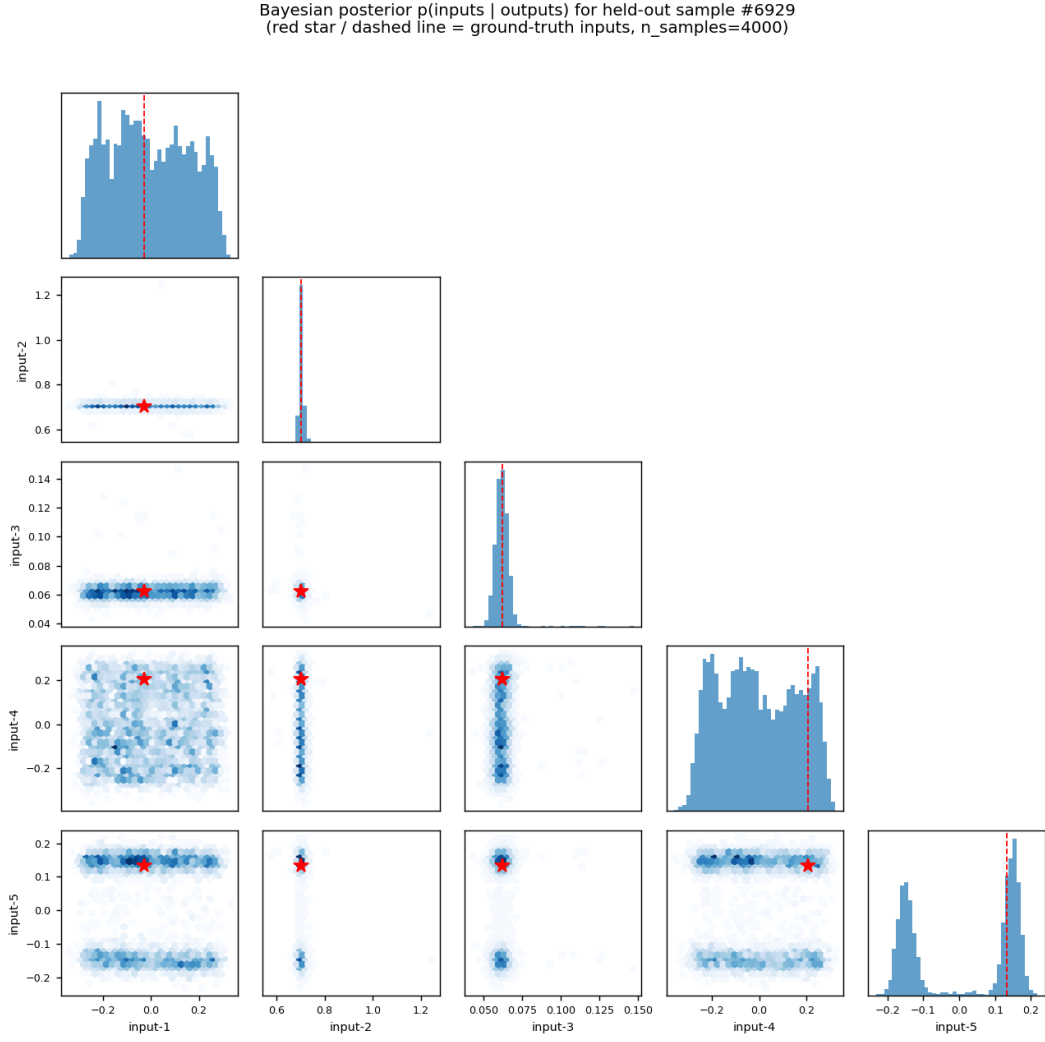


Figure 5: Bayesian posterior $p(\text{inputs} \mid \text{outputs})$ for one held-out sample. The ground-truth input vector (red star / dashed line) falls in a high-density region. The posterior is sharp on **input-2** and **input-3**, broad on **input-1** / **input-4**, and *bimodal* on **input-5** — a genuine physical degeneracy that the forward map cannot resolve.

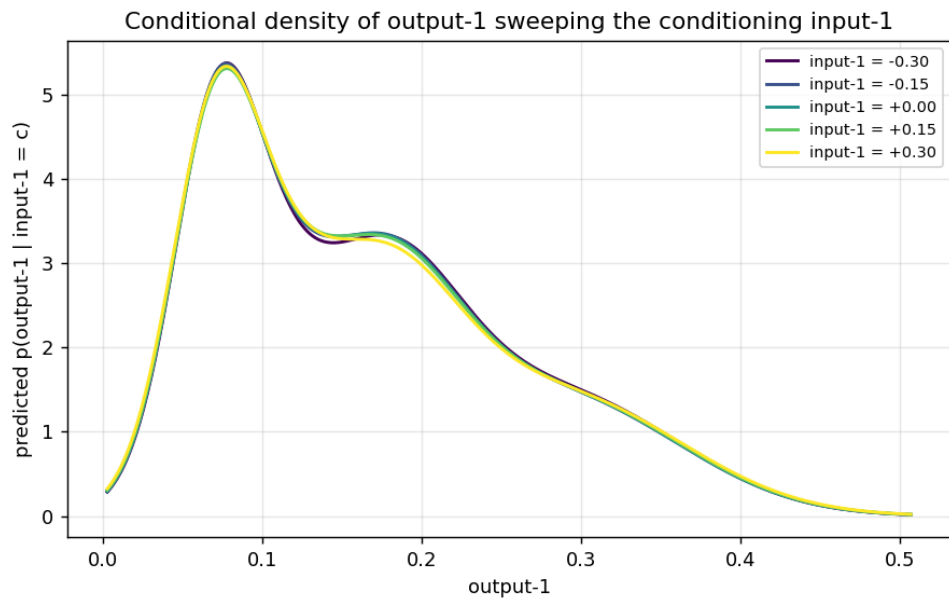


Figure 6: Sweep of `input-1` from low (purple) to high (yellow) values and the resulting predicted conditional density of `output-1`.